



Laboratorio 02.1: Explicación del código

Repositorio, entorno Conda, arquitectura modular y despliegue

Dr. Juan Bekios Calfa

Machine Learning

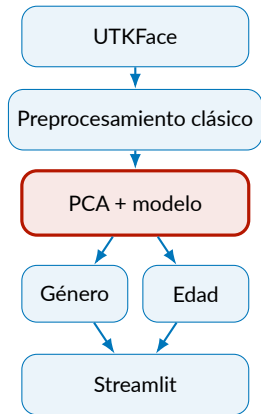
Universidad Católica del Norte

Propósito de esta presentación

- Explicar la estructura del repositorio Lab02-ML-2026-01.
- Relacionar el código con el objetivo del laboratorio: **clasificación de género y guía de regresión de edad** con UTKFace.
- Mostrar cómo preparar el proyecto desde cero: **clonar, crear un entorno Conda con YAML y ejecutar** entrenamiento y app visual.
- Describir qué hace cada módulo, evitando mostrar demasiado código.
- Visualizar la arquitectura del laboratorio usando diagramas TìkZ.

Contexto del laboratorio

- El laboratorio trabaja con **UTKFace** usando imágenes faciales ya recortadas y alineadas.
- La tarea principal implementada es **clasificación de género** con **PCA + GaussianNB**.
- La tarea de **regresión de edad** queda preparada como guía para que el estudiantado complete esa parte.
- El cierre metodológico considera **CRISP-DM** y una fase de **deployment** con Streamlit.



Qué implementa hoy el repositorio

Ya está implementado

- Carga del dataset y lectura de etiquetas desde el nombre del archivo.
- Preprocesamiento facial clásico: escala de grises, resize, ecualización, máscara oval y vectorización.
- Entrenamiento del clasificador de género con selección de componentes PCA por validación cruzada.
- Evaluación con métricas y generación de artefactos para análisis.
- Una app visual mínima en Streamlit para la fase de deployment.

Queda pendiente para estudiantes

- Completar la regresión de edad en `src/regression.py`.
- Integrar un detector de caras real en la app visual.
- Cargar ambos modelos y producir predicciones por rostro detectado.

Paso 1: clonar el repositorio

```
git clone https://github.com/jbekios/Lab02-ML-2026-01.git  
cd Lab02-ML-2026-01
```

- `git clone` descarga el repositorio completo en el equipo local.
- Al entrar a la carpeta, ya se tiene la estructura base: `main.py`, `main_visual.py`, `requirements.txt` y la carpeta `src`.
- Esta organización ayuda a separar entrenamiento, inferencia, visualización y configuración.

Paso 2: proponer un `environment.yml` para Conda

```
name: lab02-ml-2026-01
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - pip
  - numpy
  - pandas
  - matplotlib
  - scikit-learn
  - joblib
  - pip:
    - opencv-python-headless
    - streamlit
```

Idea clave

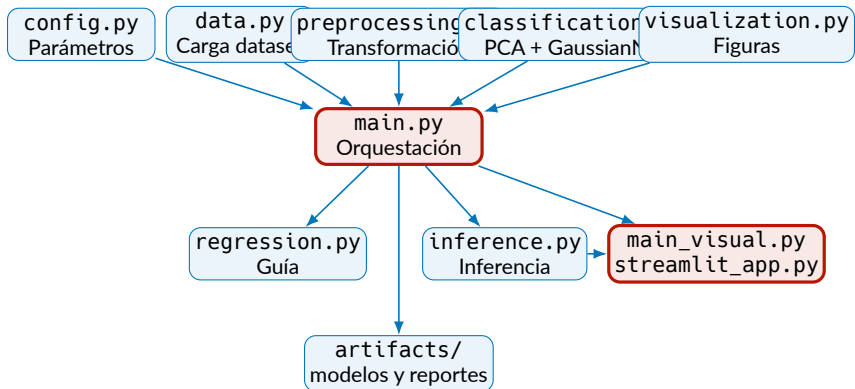
El repositorio trae `requirements.txt`. Este archivo YAML es una forma cómoda de llevar esas dependencias a un entorno Conda reproducible.

Paso 3: crear, activar y ejecutar

```
conda env create -f environment.yml
conda activate lab02-ml-2026-01
python main.py --dataset-dir "/ruta/a/UTKFace"
streamlit run main_visual.py
```

1. Se crea el entorno a partir del YAML.
2. Se activa el intérprete correcto.
3. `main.py` entrena y genera artefactos.
4. `main_visual.py` abre la app de Streamlit.

Arquitectura general del proyecto



`main.py`: el orquestador principal

Responsabilidad central

Coordina todo el entrenamiento del clasificador de género desde consola.

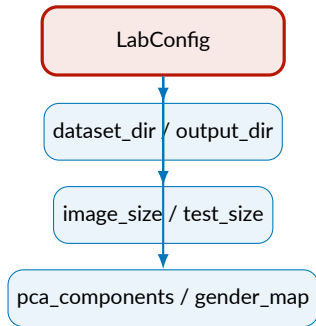
1. Lee argumentos con `parse_args()`.
2. Construye la configuración con `build_config()`.
3. Carga y preprocesa el dataset.
4. Separa entrenamiento y prueba.
5. Entrena PCA + GaussianNB.
6. Evalúa el modelo y guarda métricas, figuras y pipeline.
7. Deja además una guía textual para la parte de regresión.

Diseño acertado

`main.py` no implementa toda la lógica interna: delega en módulos especializados. Eso hace que el flujo sea más claro y mantenible.

config.py: configuración del laboratorio

- Define la clase LabConfig como dataclass.
- Centraliza rutas, tamaño de imagen, test_size, random_state, componentes PCA y mapa de género.
- Tiene propiedades útiles para models_dir, reports_dir y figures_dir.
- ensure_output_dirs() crea la estructura de salida automáticamente.



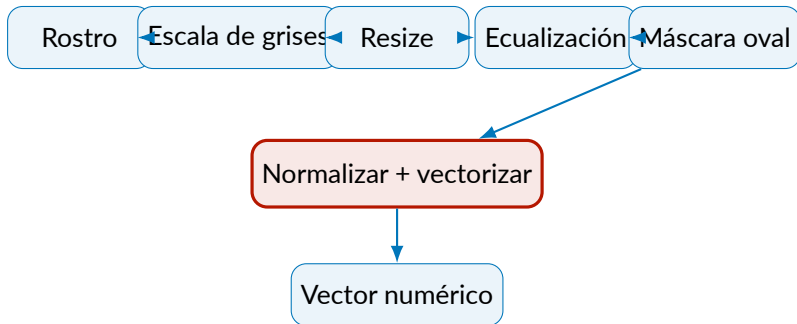
data.py: desde archivos a matriz de aprendizaje

- DatasetBundle agrupa X , etiquetas de género, etiquetas de edad, rutas de archivos y archivos omitidos.
- `parse_filename()` extrae edad y género desde nombres del tipo `edad_genero_raza_fecha.jpg`.
- `iter_image_files()` busca imágenes válidas en la carpeta del dataset.
- `build_dataset()` recorre el dataset, aplica preprocesamiento y construye la matriz principal.
- `dataset_to_dataframe()` genera una tabla simple para exportar un resumen del conjunto.

Lectura metodológica

Aquí se concreta el paso CRISP-DM de **Data Understanding + Data Preparation**: los archivos crudos se transforman en datos listos para modelar.

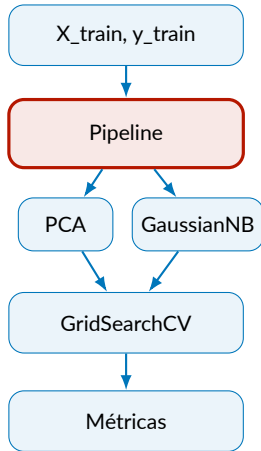
preprocessing.py: el corazón del preprocesamiento



`build_oval_mask()` genera la máscara. `preprocess_face_array()` procesa arreglos ya cargados. `preprocess_image_path()` agrega la lectura desde disco.

classification.py: entrenamiento y evaluación

- Define tres dataclasses: DatasetSplit, TrainingResult y ClassificationMetrics.
- `split_dataset()` mantiene la proporción de clases con `stratify=y_gender`.
- `build_gender_classifier()` crea el pipeline PCA + GaussianNB.
- `resolve_cv_folds()` y `resolve_pca_components()` evitan configuraciones imposibles para el tamaño real del conjunto.
- `train_gender_classifier()` usa `GridSearchCV` para elegir el mejor número de componentes.



regression.py: guía intencionalmente incompleta

¿Por qué es importante este archivo?

No es un error del repositorio: está diseñado como parte del laboratorio para que el estudiantado complete la tarea de regresión de edad.

- AgeRegressionGuide resume el camino de trabajo.
- Se sugiere reutilizar la misma matriz X ya preprocesada.
- La interfaz propone construir PCA + LinearRegression.
- También sugiere usar GridSearchCV con `neg_mean_absolute_error`.
- Las funciones `build_age_regression_pipeline()`, `train_age_regressor()`, `evaluate_age_regressor()` y `save_age_regressor()` están como TODOs didácticos.

inference.py: consistencia entre entrenamiento y uso real

- `load_gender_pipeline()` recupera el modelo entrenado desde disco con `joblib`.
- `GenderPrediction` encapsula el resultado para mostrarlo de forma legible.
- `predict_gender_from_face()` vuelve a aplicar **el mismo preprocesamiento** antes de predecir.

Principio fundamental

El modelo solo tiene sentido en producción si el rostro nuevo pasa por exactamente el mismo pipeline usado durante el entrenamiento.

Lectura conceptual

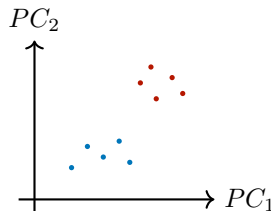
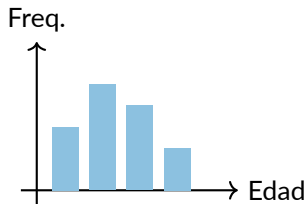
Este módulo conecta **modeling** con **deployment**: evita que la inferencia use entradas distintas a las que el modelo aprendió.

visualization.py: análisis reproducible

1. `save_dataset_distribution_figure()` resume distribución de género y edad.
2. `save_confusion_matrix_figure()` guarda la matriz de confusión.
3. `save_pca_projection_figure()` proyecta el conjunto a 2D para inspección visual.

Interpretación didáctica

Estas figuras no son decoración. Ayudan a discutir si el dataset está balanceado, cómo se comporta el clasificador y qué estructura aparece al proyectar los datos.



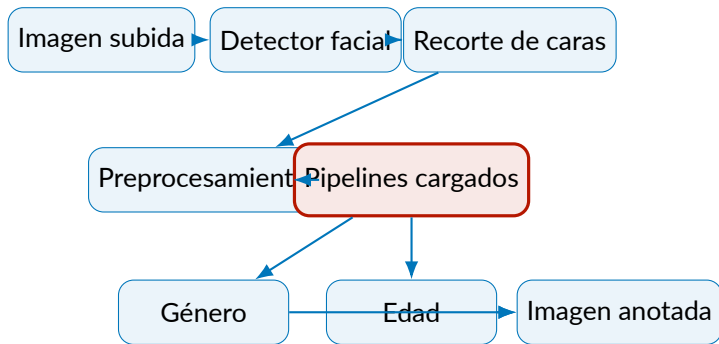
`main_visual.py` y `streamlit_app.py`

- `main_visual.py` es solo un punto de entrada mínimo: importa `run_app( )` y lo ejecuta.
- `streamlit_app.py` define la interfaz visual base.
- La app actual permite subir una imagen, verla en pantalla y explicar dónde conectar el detector de caras y los modelos entrenados.
- Por diseño, la predicción aún no se ejecuta: primero debe completarse la etapa de detección facial y la regresión de edad.

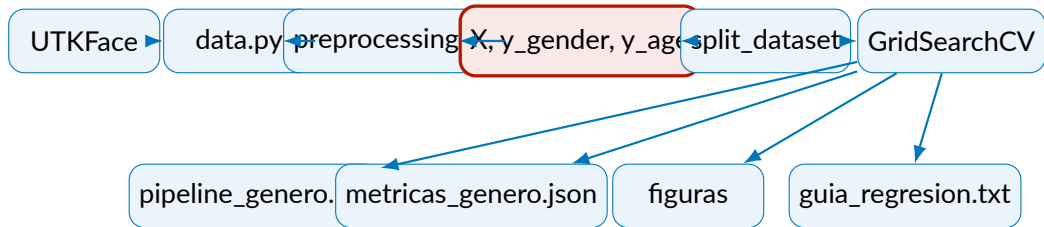
Buena decisión pedagógica

La app no oculta el trabajo pendiente. Deja explícita la frontera entre un prototipo visual y un despliegue realmente funcional.

Arquitectura de deployment propuesta



Flujo completo de ejecución



Archivos que se generan al entrenar

Modelos y reportes

- `pipeline_genero.pkl`
- `metricas_genero.json`
- `resumen_dataset.csv`
- `guia_regresion.txt`

Figuras

- `distribucion_dataset.png`
- `matriz_confusion_genero.png`
- `proyeccion_pca_genero.png`

Lectura rápida

Todos estos artefactos quedan bajo la carpeta `artifacts/` para separar claramente código fuente de salidas experimentales.

Lectura pedagógica final

- El repositorio está pensado para enseñar **flujo completo**, no solo para ejecutar un script.
- Cada módulo separa una responsabilidad: configuración, datos, preprocesamiento, modelado, inferencia, visualización y despliegue.
- La clasificación de género queda completamente implementada como línea base reproducible.
- La regresión y la detección facial se dejan como trabajo formativo para completar la fase de deployment.
- El uso de Conda + YAML facilita reproducibilidad, instalación limpia y trabajo en distintos equipos.

Mensaje final

La fortaleza del laboratorio no está solo en el modelo, sino en que el código ya está organizado para ser leído, extendido, evaluado y desplegado.

Consultas